



INDIA.RUNS

Build what next India runs on

Team Name : Hyperion

Team Leader Name : Nitanshu Tak

Problem Statement : Data & AI Challenge — Intelligent Candidate Discovery

Solution Overview

- Prism is a five-layer hybrid ranking engine that scores all 100,000 candidates against the Senior AI Engineer JD and outputs an explainable top-100 shortlist. It combines semantic fit (career-history meaning, not keywords), behavioral trust (response rate, recency), structural disqualifier rules, and a hard honeypot gate — fused into one score with full reasoning per candidate.
- Most systems pick either embeddings or rules. Prism runs both independently and on purpose: semantic similarity alone would rank a buzzword-heavy “Marketing Manager” profile highly — exactly the trap named in Redrob’s own JD. Our rule layer catches what embeddings miss; behavioral signals catch availability that skill-matching ignores entirely.

JD Understanding & Candidate Evaluation

- Layer 1 decomposes the JD into a structured object: 4 must-haves (production embeddings retrieval, vector DB/hybrid search, strong Python, ranking-evaluation experience), 5 nice-to-haves, and 8 explicit hard disqualifiers – including pure-research-only, consulting-only careers (with Redrob's own product-company exception correctly applied), title-chasers, and framework tourists.
- Beyond keyword matching, we read career-history narrative for substance (semantic fit), apply structural rules on title/tenure/company patterns (catches keyword-stuffed profiles), and weight 4 of the 23 behavioral signals – response rate, login recency, interview completion, open-to-work status – as a trust multiplier on the final score.

Ranking Methodology

- Five layers per candidate: JD requirements (Layer 1) feed a semantic scorer (Layer 2) and rule engine (Layer 3) in parallel; a trust multiplier (Layer 4) and honeypot gate (Layer 5) run alongside; Score Fusion combines all four into a ranked, tie-broken top 100.
- TF-IDF + Truncated SVD (latent semantic analysis) for fit scoring – chosen over a sentence-transformer model to avoid PyTorch/network dependency risk inside Redrob's sandbox. Rule-based pattern matching for structural checks. No hosted LLM calls anywhere in the ranking path.
- $\text{final_score} = \text{semantic_fit} \times \text{trust_multiplier} \times (1 - \text{disqualifier_penalty})$ – multiplicative, so a flagged or unavailable candidate is pulled down proportionally, not just nudged. Honeypots are hard-excluded before this formula runs.

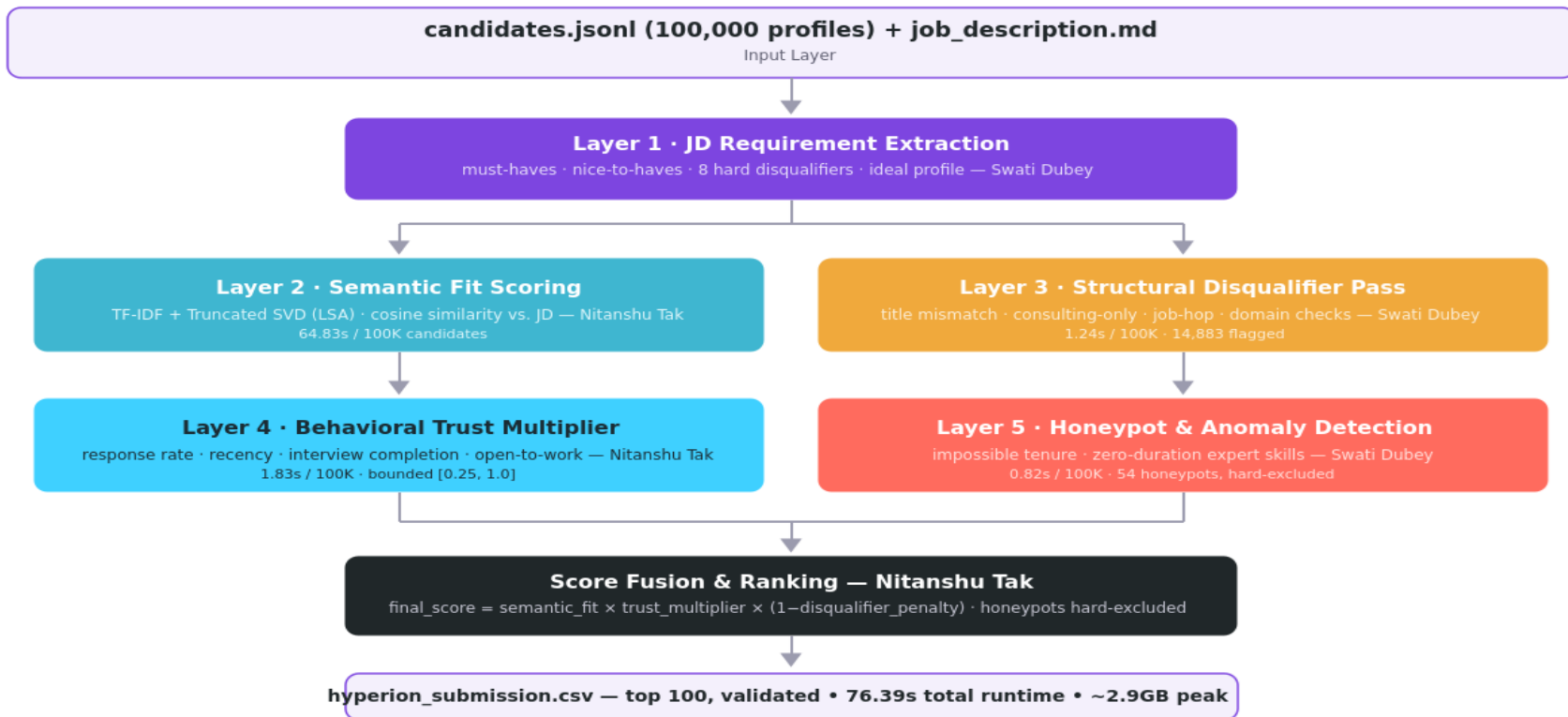
Explainability & Data Validation

- Every one of the top 100 gets a 1–2 sentence reasoning string built directly from that candidate's own computed sub-scores: years of experience, matched JD requirements by name, and behavioral-trust context — never a template, never generic boilerplate.
- No LLM call generates reasoning — it is impossible to hallucinate, by construction. Every value quoted in a reasoning string (years, title, matched requirement, response rate) is read directly from that candidate's own record or this pipeline's own computed scores, never invented.
- Layer 5 scans every profile for internal contradictions — impossible tenure (duration math vs. stated dates) and impossible skill proficiency (“expert” at zero months). Confirmed honeypots are hard-excluded before ranking. On the real 100K dataset: 54 honeypots detected, 0% in our final top 100.

End-to-End Workflow

- candidates.jsonl (100,000 profiles) + job_description.md load in.
- Layer 1 extracts structured JD requirements (must-haves, disqualifiers, ideal profile).
- Layers 2–5 run independently: semantic fit, structural rules, trust multiplier, honeypot detection.
- Score Fusion combines all four signals, hard-excludes honeypots, sorts & tie-breaks to the top 100.
- Explainability Engine writes grounded reasoning for each of the top 100, from real sub-scores.
- hyperion_submission.csv is written and validated — in 76.4 seconds, on the real 100K dataset.

System Architecture



Results & Performance

- Top 100 score range 0.380–0.480 with a smooth, properly sorted distribution. 0 candidates carry a visible structural flag — the genuinely best-fit candidates don't trip our own keyword-stuffer or consulting-only rules, a reassuring consistency check across independent layers, not a gap.
- Measured on the real 100,000-candidate file: 76.39 seconds total runtime (of a 300s/5-min budget) and ~2.9GB peak memory (of a 16GB budget). CPU-only throughout, zero network calls, zero GPU dependency — comfortable margin under every official limit.

Technologies Used

- Python 3.12, Pandas/NumPy for vectorized ops across 100K rows, scikit-learn (TF-IDF + Truncated SVD) for semantic fit. No PyTorch, no GPU, no paid API – chosen specifically to eliminate dependency-failure risk inside Redrob's sandboxed reproduction container. Total infrastructure cost: ₹0.

Submission Assets

GitHub: github.com/swaskiee/Prism-AI-Hiring-Intelligence-Platform | Live dashboard:
[dashboard/index.html](#) | Validated output: [hyperion_submission.csv](#)



INDIA.RUNS

Build what next India runs on

THANK YOU